Open in app ↗

Medium Search Write

Welcome Offer Access to everything. Now 30% off. Upgrade now

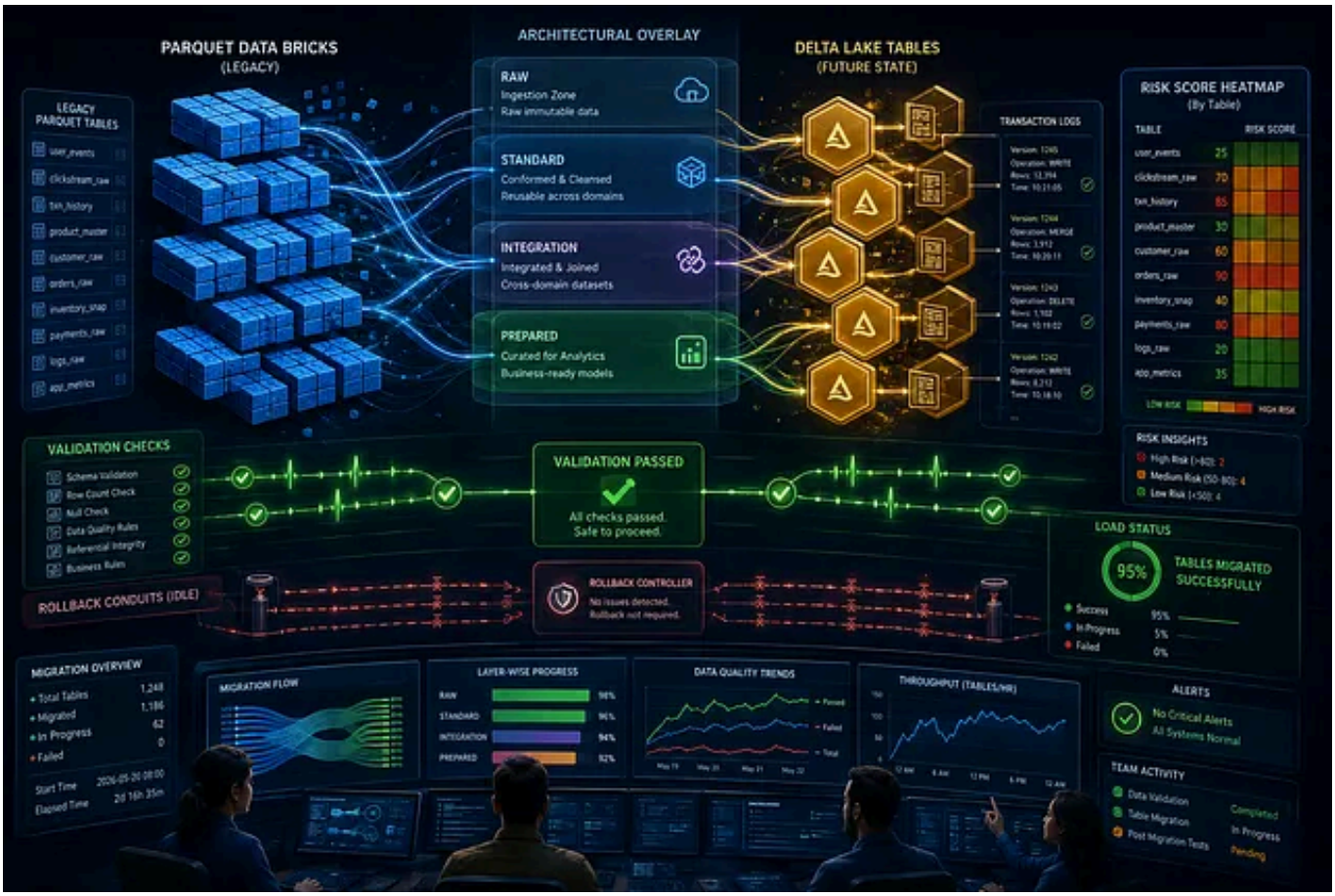
Databricks + Big Data + Data Engineering + Data + Apache Spark +

From Parquet to Delta: How We Migrated 840 Databricks Tables Without Breaking Production



Anurag Sharma 34 min read · 1 hour ago

A migration journey — the dependencies, failures, and decisions nobody talks about



In this Blog, We walk through the engineering reality of migrating 840 external Parquet tables to Delta Lake in Databricks. Forget the one-line `CONVERT TO DELTA` command. Here, we focus on managing the hidden complexity: discovering dependencies, assessing risk, validating data integrity, handling diverse write patterns, and designing a production-grade cutover.

You will learn a systematic, metadata-driven framework used by experienced teams—covering everything from building a risk-scoring inventory to planning rollback strategies. It's the blueprint for executing a large-scale platform modernization without breaking live pipelines and reports.

1. The Question That Started Everything

We had 840 Parquet tables. The platform worked. Pipelines ran every night. Reports rendered every morning. Stakeholders were, broadly speaking, happy.

Then someone asked a simple question in a planning meeting:

“Why don’t we just convert them all to Delta?”

It was a reasonable question. Databricks had been nudging the ecosystem toward Delta for years. The documentation increasingly assumed Delta as the default. Unity Catalog was becoming the governance standard. The engineering team had already been writing new tables in Delta for over a year. The older Parquet tables were beginning to feel like legacy debt — not broken, but increasingly out of place.

And on the surface, the answer seemed simple. Delta Lake has `CONVERT TO DELTA`. You point it at a Parquet table, run the command, and you're done. How hard could it be?

Very hard, as it turns out. But not for the reasons you might expect.

The hardest part of this migration was never the format conversion. The hardest part was discovering what depended on each table, how it was written, who consumed it, and what would silently break when it changed. The format conversion took minutes. The dependency analysis took weeks.

This article is about what we actually did — the planning, the framework, the failures, the trade-offs, and the lessons. If you are staring at a Databricks environment full of Parquet tables and wondering how to get to Delta without destroying your production platform, this is written for you.

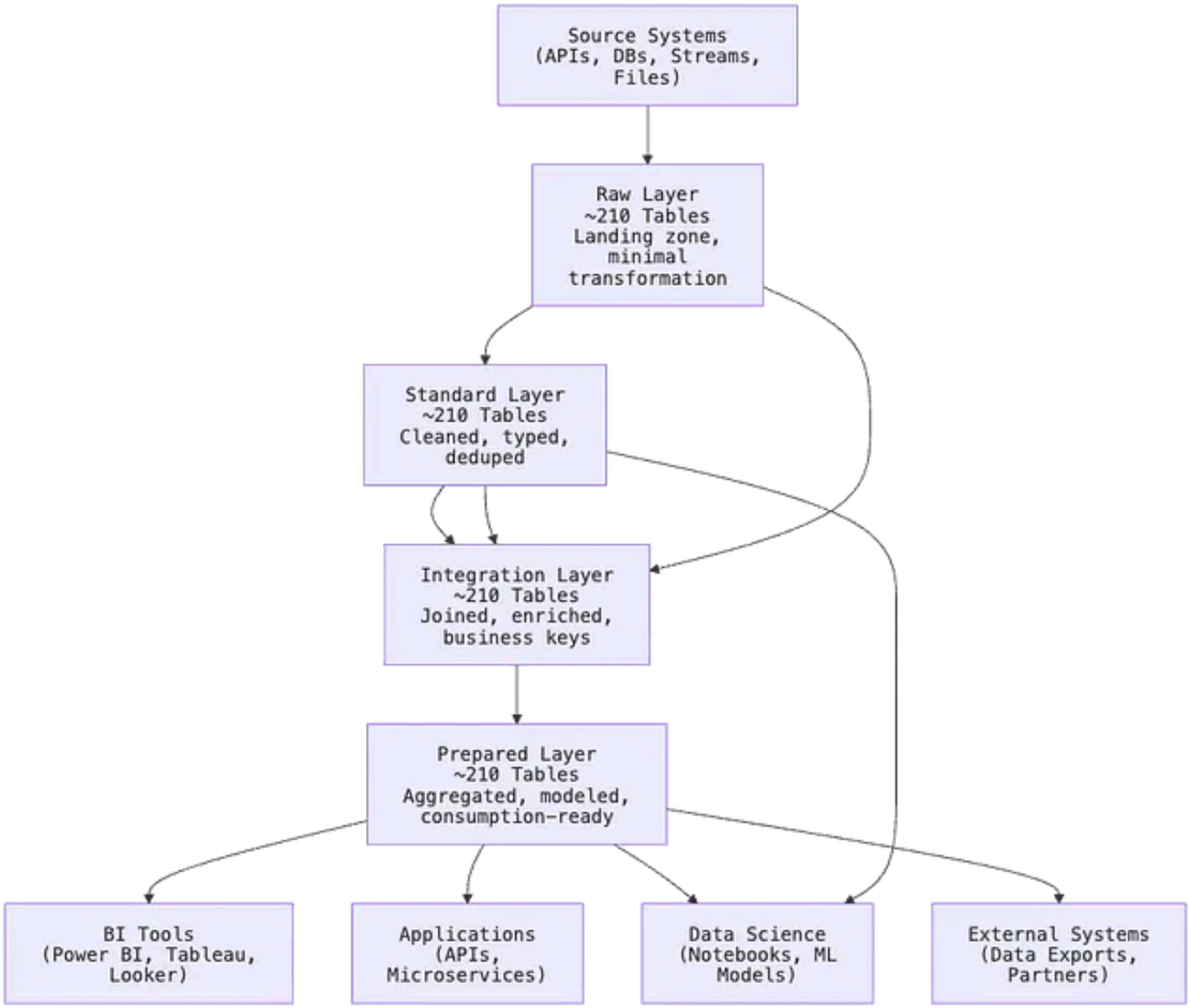
2. Our Starting Point: 840 External Parquet Tables

Let me make the scale concrete.

Layer	Table Count
Raw	~210
Standard	~210
Integration	~210
Prepared	~210
Total	~840

Our layers. Roughly symmetric. Each layer built on top of the previous one. At first glance, this looks like a clean, well-organized migration: start at Raw, work your way up to Prepared, done.

Here’s what that structure actually looks like in an enterprise environment:



Clean in a diagram. Messy in reality. Some Standard tables were consumed directly by **data science** teams. Some Integration tables had feeds going to **external partners**. Some Raw tables had been retrofitted into application-level reads because someone needed data fast three years ago and **nobody removed the reference afterward.**

The real dependency graph looked less like a waterfall and more like a graph database problem.

And that was the first major realization:

> 840 tables does not mean 840 independent migrations. The real problem is:
>
> Tables × Producers × Consumers × Dependencies × Write Patterns × Validation Requirements

That multiplication is what makes a Parquet-to-Delta migration a platform modernization program, not a file-format conversion.

3. Parquet vs. Delta — The Engineering Distinction

Before I explain what we did, let me explain why the distinction between Parquet and Delta matters operationally. Not for beginners. For engineers who already use both and want to understand the architectural implications at scale.

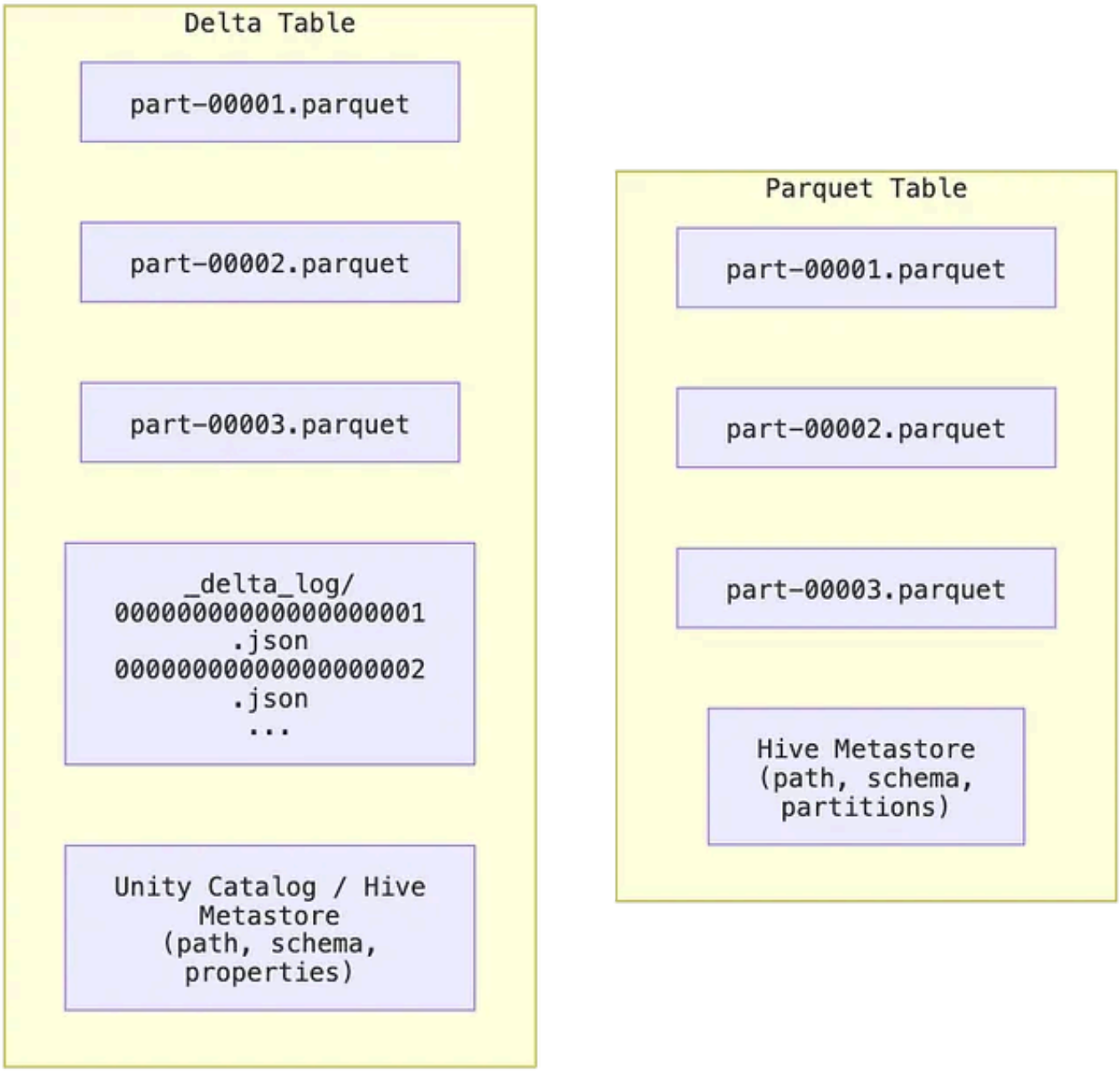
What Parquet Actually Is

Parquet is a columnar file format. It's excellent at what it does: efficient compression, columnar pruning, predicate pushdown, and wide ecosystem support. When you write a "Parquet table" in Databricks, you are writing files to a storage path and registering that path in the Hive Metastore. The table is a logical view over files.

This model works well for batch workloads that do full reads or partition reads. It starts to break down when you need mutations, concurrent writes, fine-grained reliability, or operational manageability at scale.

What Delta Actually Is

Delta is not a replacement for Parquet at the file level. The underlying storage is still Parquet files. What Delta adds is a transactional table layer on top of those files.



The `_delta_log` directory is the key. Every operation on a Delta table — write, update, delete, merge, schema change — is recorded as a JSON commit in the transaction log. This log is what gives Delta its characteristics:

Capability	Parquet	Delta
ACID transactions	❌	✅
Concurrent writes	Risky / undefined	✅ (serializable)
UPDATE / DELETE	❌ (rewrite files manually)	✅
MERGE / Upsert	❌	✅
Schema enforcement	Optional / weak	✅
Schema evolution	Manual	✅ (controlled)
Time travel	❌	✅
Change Data Feed	❌	✅
Streaming + batch	Awkward	✅ (unified)
Data recovery	Restore files manually	✅ (RESTORE command)
Audit trail	None	✅ (transaction log)
Data skipping	Partition pruning only	✅ (column statistics)
OPTIMIZE / VACUUM	Manual file management	✅

The critical point is not that Delta has more features. The critical point is that at scale — hundreds of tables, dozens of teams, concurrent producers and consumers, regulatory requirements — these capabilities have operational consequences.

A Parquet table with five writers and no coordination is a corruption incident waiting to happen. A Parquet table that needs row-level deletes requires a full partition rewrite. A Parquet table that needs time travel needs manual snapshot management. These are not theoretical problems. They are problems we hit in production.

4. Should We Migrate at All?

This is the question the team asked before writing a single line of migration code. I'd argue this is the most important question in the entire article.

The honest answer is: not every table should be migrated, and not all at once.

The Case Against Blind Migration

Let me be direct about the risks:

Migration effort is real. Eight hundred and forty tables is not a weekend project. Each table needs assessment, baseline capture, conversion, validation, consumer testing, and cutover. Done properly, that's hours of engineering effort per table, not minutes.

Regression risk is asymmetric. If a migration goes well, nobody notices. If it goes wrong, a downstream BI report shows wrong numbers and a very unhappy stakeholder calls your manager. The downside risk is much more visible than the upside.

Hard-coded Parquet paths exist everywhere. In mature Databricks environments, you will find notebooks that read directly from `abfss://container@storage.dfs.core.windows.net/raw/tablename/` rather than using the catalog table name. These do not break gracefully when you convert the table.

External consumers. Data science teams, external applications, partner integrations, and exports may read directly from storage. They have no awareness of your migration.

Historical and archive tables. Some Raw tables exist as permanent audit records. They are written once and never touched again. Converting them to Delta adds overhead with near-zero operational benefit.

Rollback is expensive. If you migrate 200 tables and discover a problem after a week of production data has landed in Delta, reverting is painful. Data that was written to Delta cannot be trivially moved back to a Parquet path without breaking the transaction log.

The Case For Migration

That said, the case for Delta in a Databricks-centric environment is strong — not because it's newer, but because of specific operational requirements that Parquet genuinely cannot meet:

ACID transactions mean you can trust that a reader never sees a half-written table. At scale with concurrent pipelines, this is not optional — it's foundational.

MERGE enables proper upsert semantics. If any of your tables need CDC, SCD Type 1, SCD Type 2, or any form of incremental update, you need MERGE. Implementing this on Parquet requires you to rewrite partitions and manage it manually. It works, but it's fragile.

Change Data Feed enables downstream consumers to efficiently process only the rows that changed. This is transformative for incremental processing patterns.

Time travel and RESTORE are your safety net. When a pipeline writes bad data, you can rewind. When a schema migration goes wrong, you can

recover. This is not hypothetical — it pays for itself the first time you need it.

Schema enforcement catches malformed data at write time. In a Parquet world, you discover schema drift when a consumer job fails at read time, often hours later.

Unity Catalog lineage — if you're on Unity Catalog, Delta tables get automatic lineage tracking. Parquet external tables don't participate in this the same way.

Databricks platform direction. Databricks is building its roadmap on Delta. Features like Predictive I/O, Liquid Clustering, and Unity Catalog governance work natively with Delta. Parquet tables are increasingly second-class citizens in the platform.

The Conclusion

> *In a Databricks-centric Lakehouse, Delta should become the default table format. But the migration should be treated as a platform modernization program, not a file-format conversion. And some tables — particularly immutable archives and tables with deep external dependencies — may warrant individual evaluation rather than mandatory inclusion.*

With that settled, let's talk about how you actually do it.

5. The Real Problem Isn't the Format

Here's the realization that changed how we approached this entire migration:

> *"At first, this looked like a storage-format migration. After we mapped the dependencies, we realized we were actually migrating an ecosystem."*

Consider a single table: `standard.customer_transactions`.

- Written by three separate ADF pipelines
- Read by two Databricks jobs in the Integration layer
- Read directly by a data science notebook
- Exported nightly to an external partner via a storage SAS token
- Partitioned by `year`, `month`, `day`
- Uses partition overwrite as its write pattern
- Has a schema that has evolved four times in the past two years

Converting this table to Delta takes thirty seconds. Understanding all of the above takes days. And every single one of those facts affects what a safe migration looks like for this specific table.

The easiest part of this migration is converting the table. The difficult part is discovering what depends on it.

This is why, before writing a single migration job, you need a metadata inventory.

6. Building the Migration Inventory

Every table in your environment must be assessed before migration begins. Not sampled. Every table. The tables you think are simple are often the ones with the hidden dependency.

Here is the metadata model we built:

```
migration_inventory_schema = {
    # Identity
    "table_name":           "string",       # catalog.schema.table
    "layer":                "string",       # raw / standard / integration / pre
    "source_path":          "string",       # physical storage path
    "table_location":       "string",       # metastore-registered location
    "format":               "string",       # parquet / delta / csv / orc

    # Structure
    "partition_columns":    "array<string>",
    "schema":               "string",       # JSON schema snapshot
    "primary_key":          "array<string>",

    # Write behavior
    "load_type":            "string",       # full / incremental / append / upse
    "write_pattern":        "string",       # overwrite / append / merge / parti
    "load_frequency":       "string",       # hourly / daily / weekly / ad-hoc /

    # Dependencies
    "producer_jobs":        "array<string>",
    "consumer_jobs":        "array<string>",
    "downstream_tables":    "array<string>",
    "upstream_tables":      "array<string>",

    # Feature requirements
    "streaming_dependency": "boolean",
    "CDC_required":         "boolean",
    "SCD_required":         "boolean",
    "SCD_type":             "string",       # type1 / type2 / none

    # Volume
    "table_size_gb":        "double",
    "record_count":         "long",
    "partition_count":      "integer",
    "file_count":           "integer",
    "last_modified":        "timestamp",

    # Business
    "criticality":          "string",       # low / medium / high / critical
    "business_owner":       "string",
    "data_classification":  "string",       # pii / confidential / internal / pu

    # Migration tracking
    "migration_wave":       "integer",
```

```
    "migration_status":     "string",       # not_started / in_progress / migrat
    "validation_status":    "string",       # not_started / pass / fail / partia
    "rollback_status":      "string",       # not_needed / available / consumed
    "risk_score":           "integer",      # 1-10

    # Notes
    "known_issues":         "string",
    "external_consumers":   "string",
    "migration_notes":      "string"
}
```

This inventory is not documentation. It is the operational database that drives your migration engine. Every migration decision — wave assignment, sequencing, strategy selection, rollback planning — should be driven from this data.

Building it requires a combination of automated discovery (querying the Hive Metastore and Unity Catalog, scanning job definitions, parsing notebook imports) and manual interviews with pipeline owners. The automated part gives you 70% of the data. The remaining 30% — the external consumers, the hard-coded paths, the undocumented reads — comes from conversations.

Those conversations are uncomfortable, but they are necessary.

7. Risk Scoring the Tables

Once you have the inventory, you can score each table for migration risk. This is what drives wave assignment.

Here is a practical scoring model:

```
┌──────────────────────────┬──────────────────────────────────────────────┬──
│ Risk Factor              │ Scoring Logic                                │ M
├──────────────────────────┼──────────────────────────────────────────────┼──
│ Downstream consumer count│ 0=0, 1-2=1, 3-5=2, 6-10=3, 10+=4             │
│ Upstream producer count  │ 0-1=0, 2-3=1, 4+=2                           │
│ Write complexity         │ full_refresh=0, append=1, partition_ow=2,    │
│                          │ merge=3, streaming=4                         │
│ CDC requirement          │ no=0, yes=3                                  │
│ Streaming dependency     │ no=0, yes=3                                  │
│ Partition complexity     │ none=0, simple=1, compound=2, skewed=3       │
│ Schema evolution history │ stable=0, minor=1, major=2                   │
│ Table size               │ lt1GB=0, 1-100GB=1, 100GB-1TB=2, 1TB+=3      │
│ Business criticality     │ low=0, medium=1, high=2, critical=3           │
│ External consumers       │ none=0, internal=1, external=3               │
│ Performance sensitivity  │ low=0, medium=1, high=2                      │
│ Rollback complexity      │ simple=0, moderate=1, complex=2              │
├──────────────────────────┼──────────────────────────────────────────────┼──
│ Total                    │                                              │
└──────────────────────────┴──────────────────────────────────────────────┴──

Normalized to 1-10: score = ceil((raw_score / 34) * 10)
```

Example: Low-Risk Table

Table: `raw.weather_feed_archive`

- 2.1 GB, 18M rows
- Written once per day, full overwrite
- No CDC, no streaming, no SCD
- One producer, zero downstream tables
- No external consumers
- Business criticality: low
- Schema stable for 3 years

Risk Score: 2/10 → Wave 1 candidate

Example: High-Risk Table

Table: `integration.customer_360`

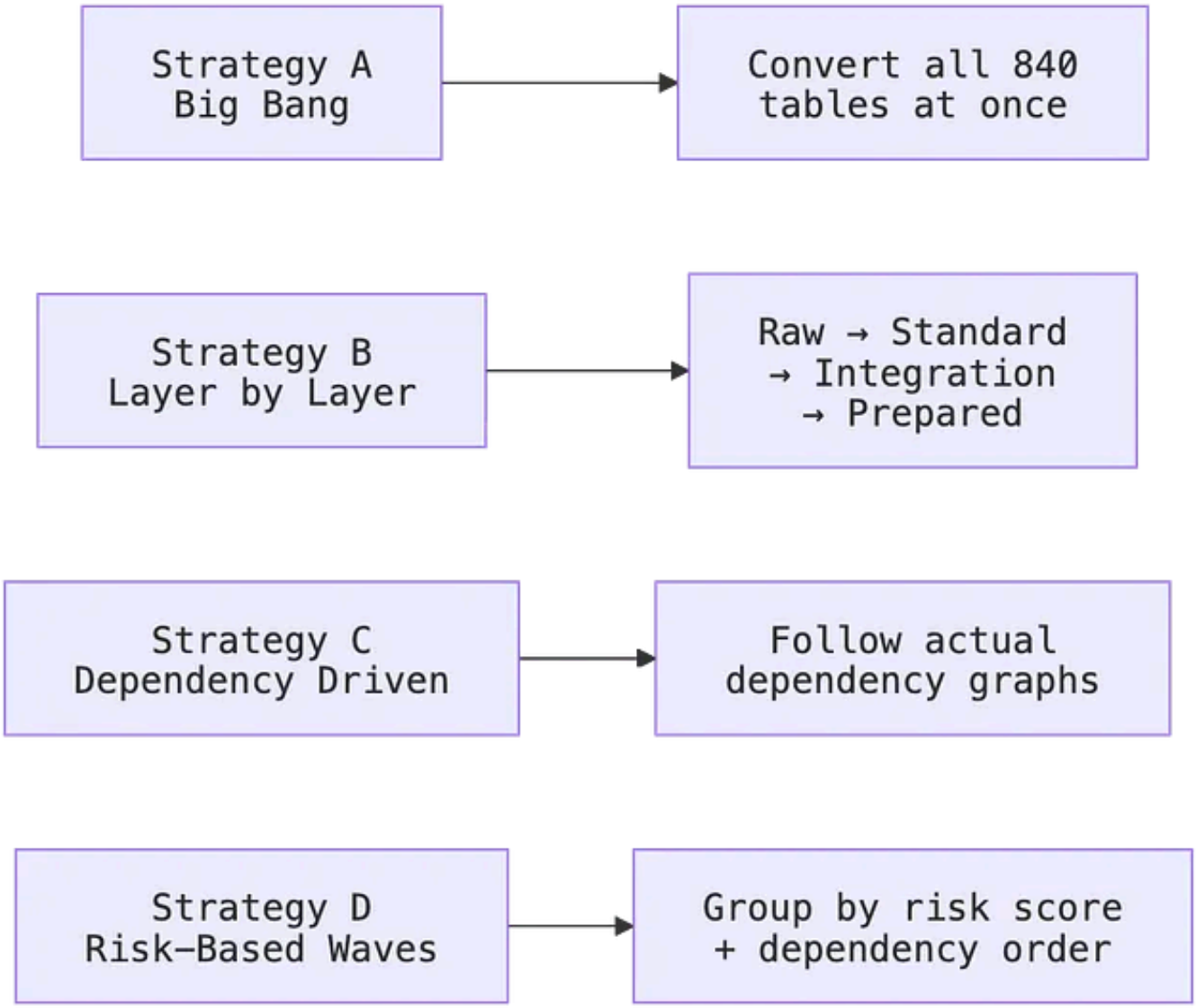
- 890 GB, 2.1B rows
- Written via MERGE from three upstream producers
- CDC required for downstream CRM sync
- Feeds five downstream tables and two BI dashboards
- Consumed by an external data export job via storage path
- Contains PII, business criticality: critical
- Schema evolved three times in two years
- Compound partition on `region, year, month`

Risk Score: 9/10 → Wave 6 or 7 candidate, requires dedicated migration plan

The risk score should not be the only criterion. Two tables with the same score may have very different dependency structures. Risk score drives wave grouping. Dependency analysis drives sequencing within waves.

8. Choosing the Migration Strategy

There is no single correct strategy for migrating 840 tables. There are, however, strategies that are clearly wrong for this scale.



Strategy A — Big Bang

Convert everything at once over a maintenance window.

Appeal: Simple to coordinate. One cutover event. No prolonged coexistence of Parquet and Delta.

Reality: With 840 tables, a big-bang conversion means 840 tables are simultaneously unvalidated. A single high-risk table failure can cascade. Rollback means reverting all 840 tables. The maintenance window required would be so large as to be politically unacceptable for most organizations.

Verdict: Avoid. The risk is too high to justify the coordination simplicity.

Strategy B — Layer by Layer

Migrate Raw, then Standard, then Integration, then Prepared.

Appeal: Respects the obvious data flow direction. Logical and easy to explain to stakeholders.

Problem: Within each layer, tables still have different risk profiles. Migrating all 210 Raw tables together is still a batch of 210 independent risks. And the real dependency graph doesn't respect layer boundaries as cleanly as the architecture diagram suggests.

Verdict: Better than big-bang, but still too coarse-grained without risk segmentation.

Strategy C — Dependency-Driven

Build a full dependency graph. Migrate tables in topological order.

Appeal: Guarantees that no table is migrated before its dependencies are stable. Theoretically correct.

Problem: In a real enterprise, the dependency graph has cycles, ambiguities, and unknown edges. Building a complete and accurate dependency graph is itself a significant project. You can use this approach for sequencing within waves, but relying on it as the primary strategy requires perfect knowledge you probably don't have.

Verdict: Use this for sequencing. Not as a standalone strategy.

Strategy D — Risk-Based Waves (Recommended)

Group tables into migration waves based on risk score. Within each wave, sequence based on dependency analysis. Start with low-risk, high-confidence tables to build tooling confidence, validate the migration framework, and demonstrate value before touching high-risk tables.

Why this works:

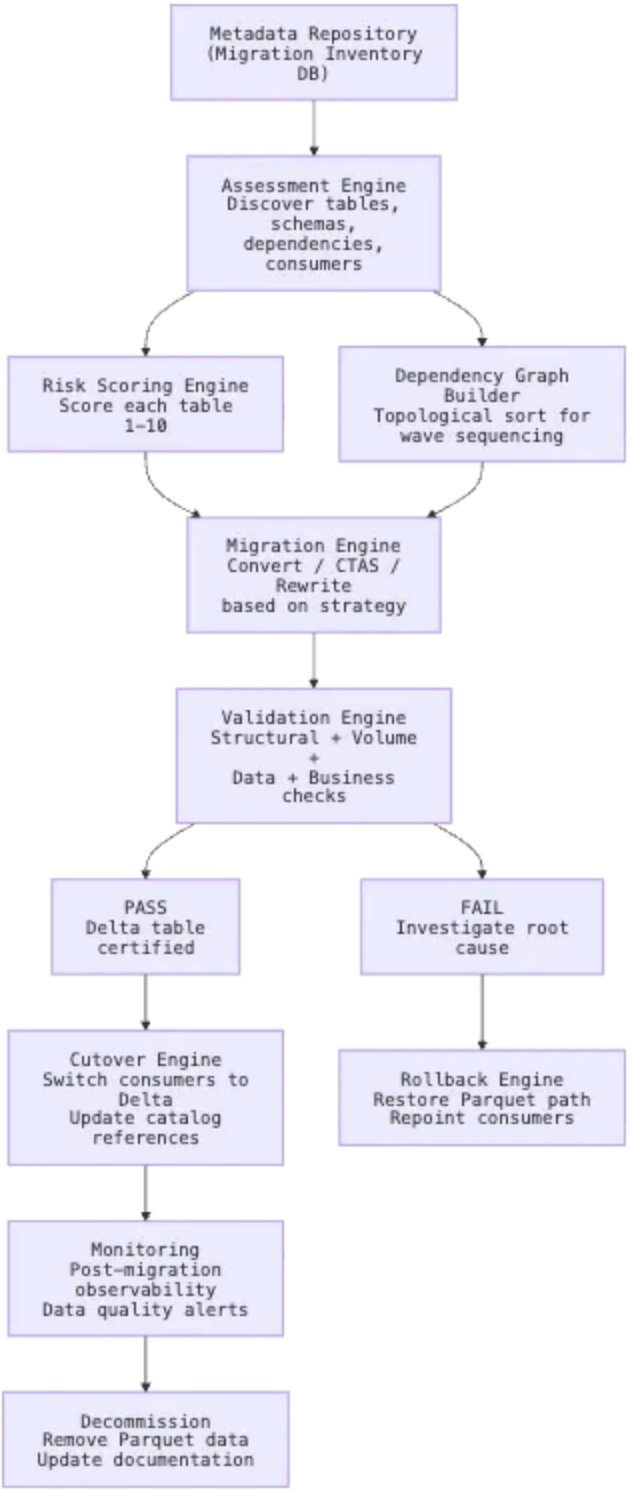
- Low-risk tables provide learning opportunities with low consequences
- The migration framework gets tested and refined on tables that matter less
- High-risk tables get migrated with a mature, validated process
- Each wave is small enough to roll back if something goes wrong
- Progress is visible and controllable

Verdict: This is the approach we recommend. The rest of this article assumes this strategy.

9. Designing the Migration Framework

A migration at this scale should not be a collection of ad-hoc notebooks. It should be a framework — a system that can be applied consistently to every table, with built-in validation, rollback, and audit.

Here's the conceptual architecture:

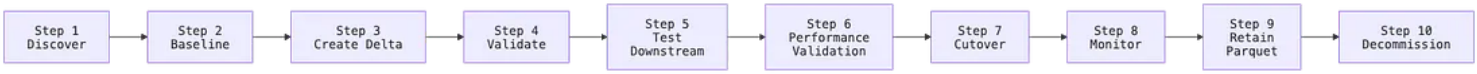


Each component is independently runnable and state-tracked in the metadata repository. If a migration job fails at the validation step, you don't re-run the entire pipeline — you resume from validation. The system knows what state each table is in.

This is important: migration state management is as important as migration logic. At 840 tables, you will have partial failures, re-runs, and resumed waves. Without state tracking, you lose visibility into what's actually been done.

10. Step-by-Step Migration — The Full Lifecycle

Let me walk through the complete lifecycle for a single table. This is what "migrating a table" actually means.



Step 1 — Discover

Before touching anything, fully characterize the table.

```
# Automated discovery — extract from Hive Metastore / Unity Catalog
def discover_table(catalog: str, schema: str, table: str) -> dict:
    spark.sql(f"USE CATALOG {catalog}")

    # Schema
    schema_df = spark.sql(f"DESCRIBE TABLE EXTENDED {schema}.{table}")

    # Partitions
    try:
        partitions = spark.sql(f"SHOW PARTITIONS {schema}.{table}").count()
    except:
        partitions = 0

    # Location and format
    detail = spark.sql(f"DESCRIBE DETAIL {schema}.{table}").collect()[0]

    # File inventory
    files = dbutils.fs.ls(detail['location'])

    return {
        "location": detail['location'],
```

```
        "format": detail['format'],
        "partition_count": partitions,
        "file_count": len(files),
        "size_bytes": sum(f.size for f in files),
        "schema": spark.table(f"{schema}.{table}").schema.json()
    }
```

Beyond the automated data, interview the pipeline owners. Ask:

- Who writes this table and when?
- Are there any jobs that read directly from the storage path?
- Does anything downstream depend on the physical file layout?
- Is there any external system that touches this storage location?
- Has the schema changed in the past year? Why?

The answers to these questions cannot be obtained from a metadata query.

Step 2 — Capture Baseline Metrics

Before any conversion, capture a complete statistical profile of the Parquet table. This becomes your ground truth for validation.

```
def capture_baseline(catalog: str, schema: str, table: str) -> dict:
    df = spark.table(f"{catalog}.{schema}.{table}")

    baseline = {
        "row_count": df.count(),
        "column_count": len(df.columns),
        "partition_count": spark.sql(
            f"SHOW PARTITIONS {catalog}.{schema}.{table}"
        ).count(),
        "schema_json": df.schema.json(),
        "captured_at": datetime.now().isoformat()
```

```
    }

    # Per-column statistics for key columns
    from pyspark.sql import functions as F

    col_stats = {}
    for col in df.columns:
        dtype = df.schema[col].dataType
        stats = {
            "null_count": df.filter(F.col(col).isNull()).count(),
            "distinct_count": df.select(col).distinct().count()
        }
        if str(dtype) in ["LongType", "DoubleType", "IntegerType", "DecimalType"
            agg = df.agg(
                F.min(col).alias("min_val"),
                F.max(col).alias("max_val"),
                F.sum(col).alias("sum_val"),
                F.avg(col).alias("avg_val")
            ).collect()[0]
            stats.update(agg.asDict())
        col_stats[col] = stats

    baseline["column_stats"] = col_stats
    return baseline
```

Store this baseline in your migration inventory. You will compare the Delta table's statistics against these numbers during validation.

Step 3 — Create the Delta Table

This step has several sub-strategies. We'll cover them in detail in the next section. For now, a representative example using CTAS (which works well for most cases):

```
# CTAS approach — creates a new Delta table from the Parquet data
def migrate_via_ctas(
    source_catalog: str, source_schema: str, source_table: str,
    target_catalog: str, target_schema: str, target_table: str,
    partition_cols: list = None
```

```
    ) -> None:

        partition_clause = ""
        if partition_cols:
            partition_clause = f"PARTITIONED BY ({', '.join(partition_cols)})"

        spark.sql(f"""
            CREATE TABLE {target_catalog}.{target_schema}.{target_table}
            USING DELTA
            {partition_clause}
            TBLPROPERTIES (
                'delta.enableChangeDataFeed' = 'false',  -- enable only if required
                'migration.source_table' = '{source_catalog}.{source_schema}.{source
                'migration.migrated_at' = '{datetime.now().isoformat()}',
                'migration.source_format' = 'parquet'
            )
            AS SELECT * FROM {source_catalog}.{source_schema}.{source_table}
        """)

        # Optimize immediately after creation
        spark.sql(f"""
            OPTIMIZE {target_catalog}.{target_schema}.{target_table}
        """)

        # Compute statistics for query optimizer
        spark.sql(f"""
            ANALYZE TABLE {target_catalog}.{target_schema}.{target_table}
            COMPUTE STATISTICS FOR ALL COLUMNS
        """)
```

Step 4 — Validate

Validation is covered in depth in Section 13. At this step, you run structural, volume, and data validation checks and compare against the baseline.

Step 5 — Test Downstream Jobs

Do not assume that a validated Delta table means all downstream jobs will work. Run them in test mode against the Delta table.

This means:

- Running producer jobs in test mode and verifying their write behavior in Delta
- Running consumer jobs against the Delta table and comparing outputs to expected results
- Checking for hard-coded path references
- Running streaming jobs that read this table and verifying checkpoint compatibility
- Testing any scheduled exports or external reads

This step is time-consuming and cannot be skipped for medium and high-risk tables.

Step 6 — Performance Validation

Compare the query performance of key workloads between Parquet and Delta. This does not mean Delta will always be faster — it means you need to know if it's significantly slower before cutover, not after.

```
-- Capture query plan and execution time for representative workloads
-- Run against both Parquet and Delta tables
EXPLAIN COST
SELECT region, SUM(revenue) as total_revenue
FROM integration.customer_transactions
WHERE year = 2025 AND month = 11
GROUP BY region;
```

Record: execution time, bytes scanned, files read, shuffle spill. If Delta is significantly slower on a key workload, investigate before proceeding.

Step 7 — Cutover

Switch consumers from the Parquet table to the Delta table. More on cutover patterns in Section 15.

Steps 8–10 — Monitor, Retain, Decommission

After cutover, monitor the Delta table in production for at least one full pipeline cycle (and ideally one full business reporting cycle). Keep the Parquet data available for rollback. Only decommission the Parquet data after the migration has been formally certified.

11. Migration Techniques — No Single Approach Fits All

There are multiple ways to technically execute the conversion. The wrong choice for a given table can create problems that are expensive to fix after cutover.

Technique 1 — `CONVERT TO DELTA`

```
CONVERT TO DELTA parquet.`abfss://container@storage.dfs.core.windows.net/path/to
PARTITIONED BY (year INT, month INT, day INT);
```

What it does: Converts an existing Parquet table in-place by creating a `_delta_log` on top of the existing Parquet files. No data is copied.

When to use: The table is external, the Parquet files are well-organized, and you want an in-place conversion without moving data.

Limitations:

- Only works on Parquet (not ORC, CSV, etc.)
- The existing file layout becomes permanent; there's no chance to optimize
- Cannot be used on managed tables
- Not compatible with tables using custom file naming conventions
- After conversion, the first OPTIMIZE run will be important for performance

Risk: If something goes wrong during conversion (e.g., the process is interrupted), you may end up with a partially converted table. Always take a snapshot of the metadata before running this.

Technique 2 — CTAS (CREATE TABLE AS SELECT)

```
CREATE TABLE catalog.schema.delta_table
USING DELTA
PARTITIONED BY (year, month)
TBLPROPERTIES ('delta.enableChangeDataFeed' = 'true')
AS SELECT * FROM catalog.schema.parquet_table;
```

What it does: Creates a new Delta table from the Parquet data, writing fresh Parquet files under a new location.

When to use: You want a clean Delta table with optimized file layout. The table location can change. You want to take the opportunity to apply a new partition strategy.

Advantages:

- Clean file layout from the start
- Opportunity to restructure partitioning
- Original Parquet data is untouched (rollback-friendly)
- Can run concurrently with the live Parquet table

Limitation: Doubles your storage during migration. For a 1 TB table, you need 1 TB of additional space during the migration window.

Technique 3 — DataFrame Write

```
# Read from Parquet, write to Delta
df = spark.table("catalog.schema.parquet_table")

df.write \
    .format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .partitionBy("year", "month") \
    .saveAsTable("catalog.schema.delta_table")
```

Technique 4 — Incremental / Partition-Level Migration

For very large tables where a full CTAS is impractical:

```
# Migrate one partition at a time
partitions = spark.sql(
    "SHOW PARTITIONS catalog.schema.large_parquet_table"
).collect()

for partition in partitions:
    year = partition['year']
    month = partition['month']

    df = spark.sql(f"""
```

```
        SELECT * FROM catalog.schema.large_parquet_table
        WHERE year = {year} AND month = {month}
    """)

    df.write \
        .format("delta") \
        .mode("append") \
        .partitionBy("year", "month") \
        .option("replaceWhere", f"year = {year} AND month = {month}") \
        .saveAsTable("catalog.schema.delta_table")

    # Validate this partition before continuing
    validate_partition(year, month)
```

When to use: Tables larger than 500 GB where a full CTAS would take too long or risk a timeout. Allows you to validate incrementally.

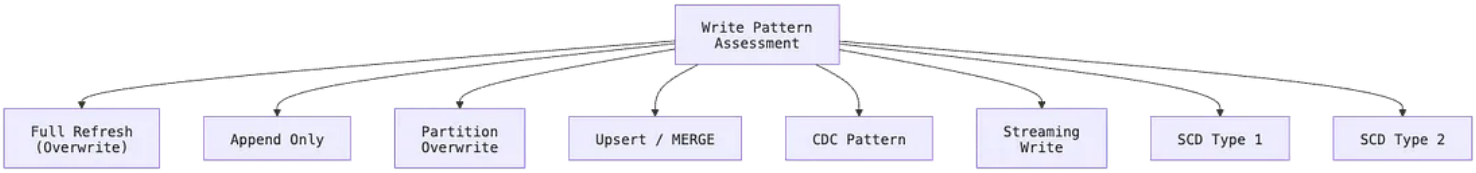


Warning: Ensure you handle metadata registration correctly and that concurrent writes don't cause conflicts during partition-level migration.

Technique	Data Copied	Original Untouched	Rollback Ease	Control	Best For
CONVERT TO DELTA	No	No	Hard	Low	Small-medium external tables
CTAS	Yes	Yes	Easy	Medium	Most cases
DataFrame Write	Yes	Yes	Easy	High	Complex transformations
Incremental/Partition	Yes	Yes	Medium	High	Very large tables

12. Write Patterns — The Most Important Section

Here is the mistake I see most often in Parquet-to-Delta migrations: teams focus entirely on reading the data and not on how the table is written. The write pattern is what determines whether your Delta migration works correctly in production.



Full Refresh (Overwrite)

The simplest pattern. Every run replaces the entire table.

Parquet behavior: Write new files to the path, delete old files.

Delta behavior:

```
df.write \
    .format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "false") \  # enforce schema
    .saveAsTable("catalog.schema.table")
```

Delta's overwrite is ACID. Readers see either the old version or the new version, never a partial state. This is a genuine improvement over Parquet, where a concurrent reader during a full overwrite can read a mix of old and new files.

Migration risk: Low. But verify that all producer jobs use the Delta writer and aren't still pointing at the raw storage path.

Append Only

New records are added; existing records are never modified.

Delta behavior:

```
df.write \
    .format("delta") \
    .mode("append") \
    .saveAsTable("catalog.schema.table")
```

Delta handles append well, but watch for small file accumulation. Parquet tables that use append often have thousands of small files. After migration, run OPTIMIZE regularly.

```
OPTIMIZE catalog.schema.table
ZORDER BY (event_timestamp);
```

Migration risk: Low to medium. The main concern is the small files problem and ensuring the OPTIMIZE/VACUUM schedule is established before cutover.

Partition Overwrite

This is one of the most common patterns and one of the most dangerous migration points.

Parquet behavior:

```
spark.conf.set("spark.sql.sources.partitionOverwriteMode", "dynamic")

df.write \
    .format("parquet") \
    .mode("overwrite") \
    .partitionBy("year", "month") \
    .save("abfss://path/to/table")
```

Dynamic partition overwrite in Parquet replaces only the partitions present in the DataFrame. This is a common pattern for incremental loads.

Delta behavior — this is where teams get burned:

In Delta, `mode("overwrite")` without `replaceWhere` overwrites the entire table, not just the partitions in the DataFrame. This is a breaking change if your producer job simply swaps "parquet" for "delta" without reviewing the write logic.

The correct Delta equivalent:

```
# Option 1: replaceWhere for explicit partition replacement
df.write \
    .format("delta") \
    .mode("overwrite") \
    .option("replaceWhere", "year = 2025 AND month = 11") \
    .saveAsTable("catalog.schema.table")

# Option 2: Enable dynamic partition overwrite (Delta 2.0+)
spark.conf.set("spark.sql.sources.partitionOverwriteMode", "dynamic")
df.write \
    .format("delta") \
```

```
    .mode("overwrite") \
    .saveAsTable("catalog.schema.table")
```

Migration risk: High. Every producer job using partition overwrite must be explicitly reviewed and updated. This is not optional. I have seen migrations that passed validation but silently corrupted production data after cutover because the producer job truncated the Delta table on the first run instead of overwriting only the target partition.

Upsert / MERGE

This is where Delta genuinely outperforms Parquet.

Parquet equivalent (painful):

```
# Read existing + new data, dedup, write back entire table
existing = spark.table("catalog.schema.parquet_table")
incoming = spark.table("staging.new_records")
merged = existing.union(incoming) \
    .dropDuplicates(["primary_key"]) \
    .filter(F.col("is_active") == True)
merged.write.mode("overwrite").parquet("path/to/table")
```

Delta MERGE (clean and transactional):

```
MERGE INTO catalog.schema.delta_table AS target
USING staging.new_records AS source
ON target.customer_id = source.customer_id
WHEN MATCHED AND source.operation = 'DELETE' THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Migration consideration: If you're migrating a table that was previously doing "merge via full rewrite" in Parquet, you now have the opportunity to implement a proper MERGE pattern. This is a genuine improvement, but it requires rewriting the producer job. Don't treat this as a pure lift-and-shift — take the opportunity to fix the write pattern.

CDC (Change Data Capture)

If a table feeds a CDC consumer (e.g., syncing changes to a downstream system or streaming consumer), you need Delta's Change Data Feed.

```
-- Enable CDF on the Delta table
ALTER TABLE catalog.schema.delta_table
SET TBLPROPERTIES ('delta.enableChangeDataFeed' = 'true');
```

```
# Consumers read the change feed
changes = spark.readStream \
    .format("delta") \
    .option("readChangeFeed", "true") \
    .option("startingVersion", 0) \
    .table("catalog.schema.delta_table")
```

Migration consideration: A Parquet table cannot produce a change feed. If your architecture requires CDC, this table must move to Delta. There is no workaround.

Streaming Writes

If a table is the target of a Spark Structured Streaming job, the migration requires particular care around checkpoints.

```
# Streaming write to Delta
df.writeStream \
    .format("delta") \
    .outputMode("append") \
    .option("checkpointLocation", "/checkpoints/table_name") \
    .table("catalog.schema.delta_table")
```

Critical: Streaming checkpoints are tied to the source path, not the logical table name. When you migrate from Parquet to Delta, existing streaming checkpoints may not transfer cleanly. In many cases, you will need to reset the checkpoint and replay from a known offset.

This is one of the highest-risk migration scenarios. Handle streaming tables individually, with dedicated testing and an agreed-upon replay strategy.

SCD Type 2

SCD Type 2 in Parquet requires complex, error-prone manual orchestration. Delta simplifies this significantly.

```
-- SCD Type 2 MERGE in Delta
MERGE INTO catalog.schema.customer_dim AS target
USING (
    SELECT
        customer_id,
        customer_name,
        email,
        address,
        current_timestamp() AS effective_start,
        '9999-12-31' AS effective_end,
        TRUE AS is_current
    FROM staging.customer_updates
) AS source
ON target.customer_id = source.customer_id
   AND target.is_current = TRUE
```

```
-- Close the existing active record
WHEN MATCHED AND (
    target.customer_name != source.customer_name OR
    target.email != source.email OR
    target.address != source.address
) THEN UPDATE SET
    target.effective_end = current_timestamp(),
    target.is_current = FALSE

-- Insert the new active record
WHEN NOT MATCHED THEN INSERT *
```

If a Parquet table is being used for SCD Type 2 with a custom implementation, migration to Delta is a significant improvement — but it also means rewriting the producer job.

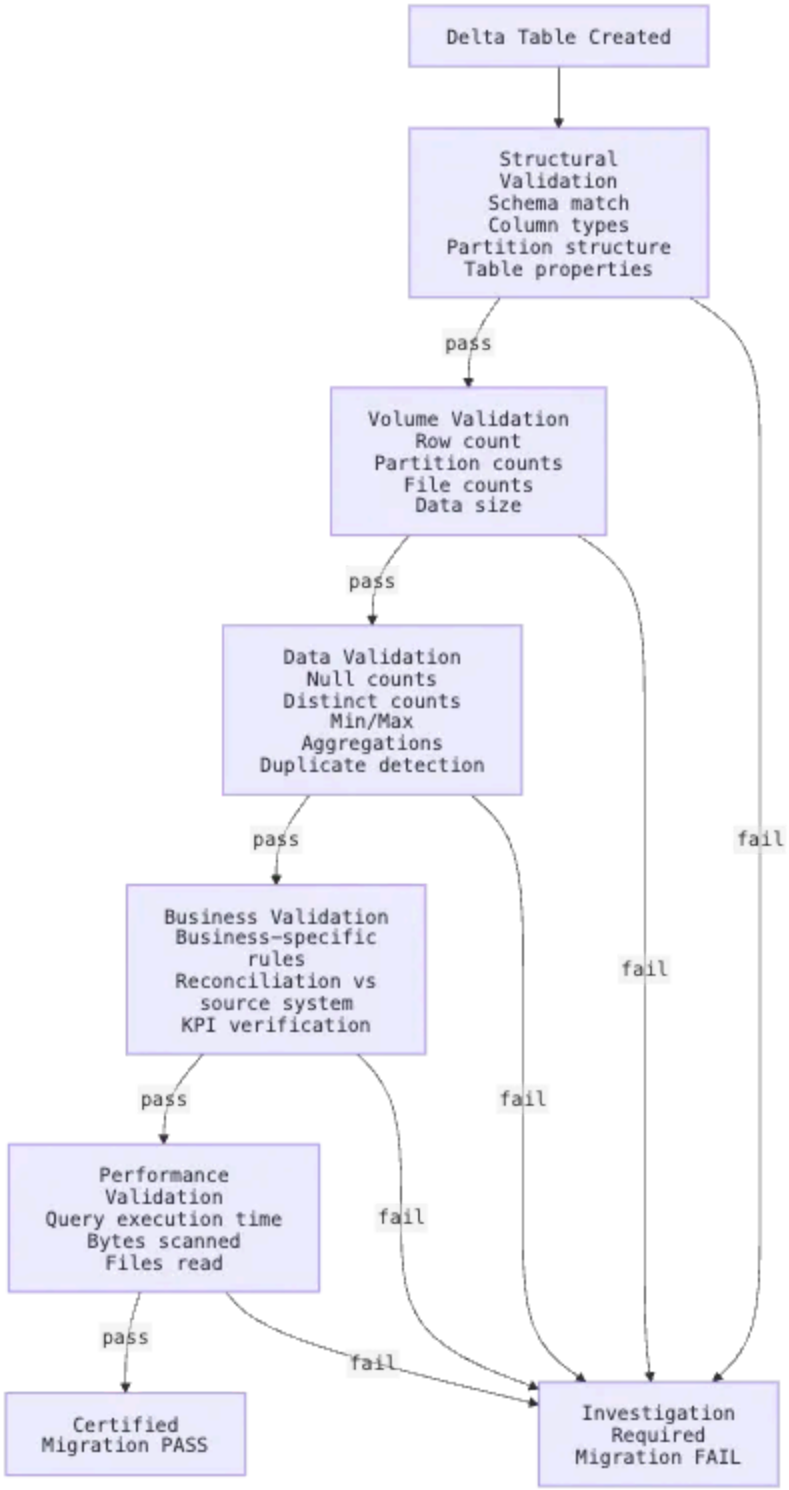
13. Data Validation — The Certification Step

I want to be direct about something: creating a Delta table successfully is not the same as completing a migration.

A migration is complete only when the Delta table has been validated, certified, and accepted as functionally equivalent to the source.
“Functionally equivalent” is a precise term. It means:

- The schema is identical (or intentionally evolved with a documented reason)
- The row count matches (or the delta is explained)
- Statistical properties of the data match
- All downstream jobs produce correct outputs
- Performance is acceptable

Here is the validation framework:



Structural Validation

```
def validate_structure(source_table: str, target_table: str) -> dict:
    source_schema = spark.table(source_table).schema
    target_schema = spark.table(target_table).schema

    source_cols = {f.name: str(f.dataType) for f in source_schema.fields}
    target_cols = {f.name: str(f.dataType) for f in target_schema.fields}

    missing_in_target = set(source_cols.keys()) - set(target_cols.keys())
    extra_in_target = set(target_cols.keys()) - set(source_cols.keys())
    type_mismatches = {
        col: (source_cols[col], target_cols[col])
        for col in source_cols.keys() & target_cols.keys()
        if source_cols[col] != target_cols[col]
    }

    return {
        "pass": not any([missing_in_target, extra_in_target, type_mismatches]),
        "missing_columns": list(missing_in_target),
        "extra_columns": list(extra_in_target),
        "type_mismatches": type_mismatches
    }
```

Volume Validation

```
def validate_volume(source_table: str, target_table: str,
                    baseline: dict, tolerance_pct: float = 0.001) -> dict:
    source_count = spark.table(source_table).count()
    target_count = spark.table(target_table).count()

    diff = abs(source_count - target_count)
    diff_pct = diff / source_count if source_count > 0 else 0

    return {
        "source_count": source_count,
        "target_count": target_count,
        "diff": diff,
        "diff_pct": diff_pct,
        "pass": diff_pct <= tolerance_pct,
        "baseline_count": baseline["row_count"],
```

```
            "baseline_match": source_count == baseline["row_count"]
        }
```

Data Validation

```
def validate_data(source_table: str, target_table: str,
                  key_columns: list) -> dict:
    source_df = spark.table(source_table)
    target_df = spark.table(target_table)

    results = {}

    for col in key_columns:
        s_stats = source_df.agg(
            F.count(F.when(F.col(col).isNull(), 1)).alias("null_count"),
            F.countDistinct(col).alias("distinct_count"),
            F.min(col).alias("min_val"),
            F.max(col).alias("max_val"),
            F.sum(col).alias("sum_val") if col in numeric_cols else F.lit(None)
        ).collect()[0]

        t_stats = target_df.agg(
            F.count(F.when(F.col(col).isNull(), 1)).alias("null_count"),
            F.countDistinct(col).alias("distinct_count"),
            F.min(col).alias("min_val"),
            F.max(col).alias("max_val"),
            F.sum(col).alias("sum_val") if col in numeric_cols else F.lit(None)
        ).collect()[0]

        results[col] = {
            "null_count_match": s_stats["null_count"] == t_stats["null_count"],
            "distinct_count_match": s_stats["distinct_count"] == t_stats["distin
            "min_match": s_stats["min_val"] == t_stats["min_val"],
            "max_match": s_stats["max_val"] == t_stats["max_val"],
            "sum_match": abs(
                (s_stats["sum_val"] or 0) - (t_stats["sum_val"] or 0)
            ) < 0.01 if s_stats["sum_val"] else True
        }

    return results
```

A Note on “Row Count Match”

Row count matching is a necessary condition for migration success, but not a sufficient one. I have seen migrations where the row count matched exactly but:

- Duplicate rows existed in the Delta table (a MERGE logic bug)
- NULL values were introduced by a schema type mismatch during conversion
- Aggregated financial values were off by 0.01% due to floating-point handling
- String encoding differences caused character corruption in a minority of rows

Row count is the first thing you check. Business validation is the last — and it’s the one that actually certifies the migration.

14. Performance — Expectations vs. Reality

Delta does not automatically make your queries faster. Let me be direct about this.

Delta provides the *infrastructure* for better performance. Whether you realize those improvements depends on how you design and maintain your tables.

What Delta Does for Performance

Data skipping: Delta maintains per-file column statistics (min, max, null count) for the first 32 columns by default. The query optimizer uses these statistics to skip files that don’t match filter predicates, reducing bytes scanned — often dramatically.

OPTIMIZE: Compacts small files into larger, more efficient files. Addresses the small file problem that accumulates in append-heavy tables.

```
-- Basic optimization
OPTIMIZE catalog.schema.table;

-- With ZOrder for frequently filtered columns
OPTIMIZE catalog.schema.table
ZORDER BY (customer_id, transaction_date);
```

Liquid Clustering (Databricks Runtime 13.3+): A more flexible alternative to Z-Ordering that works incrementally and handles skewed data better. For new tables and migrated tables being restructured, consider Liquid Clustering over static partitioning.

```
CREATE TABLE catalog.schema.table
CLUSTER BY (customer_id, region)
AS SELECT * FROM source_table;
```

Predictive I/O (Databricks): On Databricks, the runtime uses machine learning to predict which files a query will access and pre-fetches them. This works only with Delta tables on Unity Catalog.

What Can Make Delta Slower

Partition design: A Parquet table with a good partition strategy migrated to Delta with the same partition strategy works fine. A table with poor partitioning (too many small partitions, high-cardinality partition keys,

partition skew) migrated to Delta doesn't improve — it just adds transaction log overhead to a poorly designed table.

Too many columns in column statistics: By default, Delta collects statistics on the first 32 columns. If your table has important filter columns beyond position 32, reorder them or explicitly configure statistics collection.

Unconfigured OPTIMIZE schedule: A Delta table that is never OPTIMIZED will accumulate small files just as badly as a Parquet table. OPTIMIZE is not automatic — you need to run it on a schedule.

```
-- Configure auto-optimization (Databricks)
ALTER TABLE catalog.schema.table
SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact' = 'true'
);
```

Transaction log overhead on very frequent writes: Tables with extremely high-frequency writes (thousands of commits per hour) will generate large transaction logs. VACUUM regularly and consider log checkpointing.

Performance Validation Approach

```
def benchmark_query(table_ref: str, query_template: str, iterations: int = 3):
    """Run a representative query multiple times and capture metrics."""
    results = []

    for i in range(iterations):
        start = time.time()
        spark.sql(query_template.format(table=table_ref)).count()
        elapsed = time.time() - start
```

```
        # Capture execution metrics from Spark UI via REST API
        # or from spark.sparkContext.statusTracker()
        results.append({"iteration": i, "elapsed_seconds": elapsed})

    return {
        "table": table_ref,
        "avg_seconds": sum(r["elapsed_seconds"] for r in results) / len(results)
        "min_seconds": min(r["elapsed_seconds"] for r in results),
        "results": results
    }

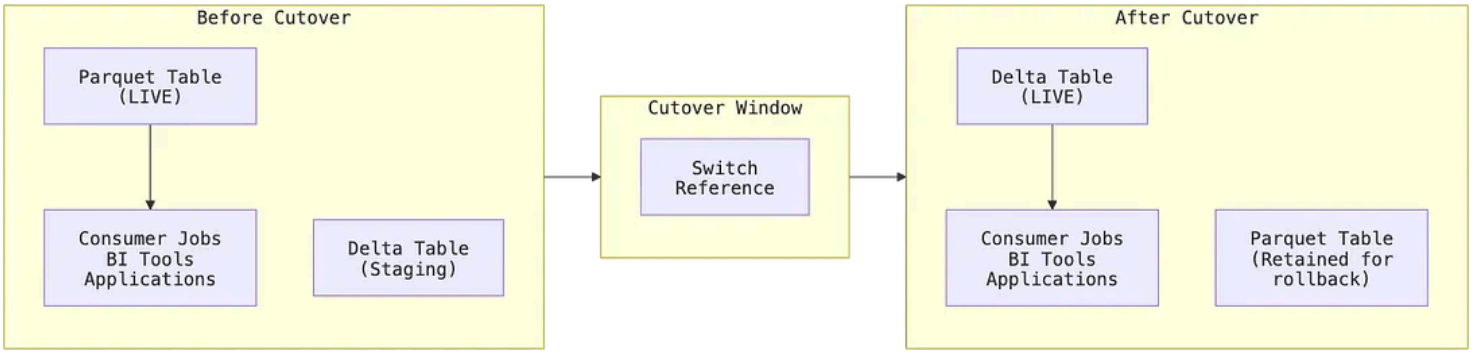
# Compare Parquet vs Delta on representative queries
parquet_perf = benchmark_query("catalog.schema.parquet_table",
                               "SELECT region, SUM(revenue) FROM {table} WHERE y
delta_perf   = benchmark_query("catalog.schema.delta_table",
                               "SELECT region, SUM(revenue) FROM {table} WHERE y

improvement = (parquet_perf["avg_seconds"] - delta_perf["avg_seconds"]) / parque
print(f"Performance change: {improvement:.1f}%")
```

If a key workload regresses significantly after migration, investigate before cutover. Common causes: missing ZOrder, partition explosion, uncollected statistics, or a filter column outside the statistics range.

15. Production Cutover

Cutover is the moment when consumers switch from the Parquet table to the Delta table. It's also the moment when things can go wrong publicly.



Cutover Patterns

Table-by-table cutover: Switch consumers one table at a time. Lowest blast radius. Highest operational effort. Appropriate for high-risk tables.

Wave-based cutover: Cut over all tables in a migration wave simultaneously. Efficient, but requires that all tables in the wave have been validated.

Dual-read: During the transition, consumers query both Parquet and Delta and compare results. Expensive, but provides a verification window before full cutover.

Consumer-by-consumer migration: Instead of switching all consumers of a table at once, migrate them one at a time. A BI dashboard migrates first. Then a downstream job. Then an application. This is safer but requires that both Parquet and Delta remain in sync — which means either the producer writes to both (dual-write) or you accept a short data consistency window.

Dual-write (use sparingly): Write to both Parquet and Delta simultaneously during transition. Ensures both are current. Adds write overhead and complexity. Use this only for the highest-criticality tables where zero downtime is required.

The Actual Cutover Moment

For most tables, cutover is a catalog operation:

```
-- Option 1: Rename tables (if using same schema)
-- Rename the old Parquet table
ALTER TABLE catalog.schema.customer_transactions
RENAME TO catalog.schema.customer_transactions_parquet_backup;
```

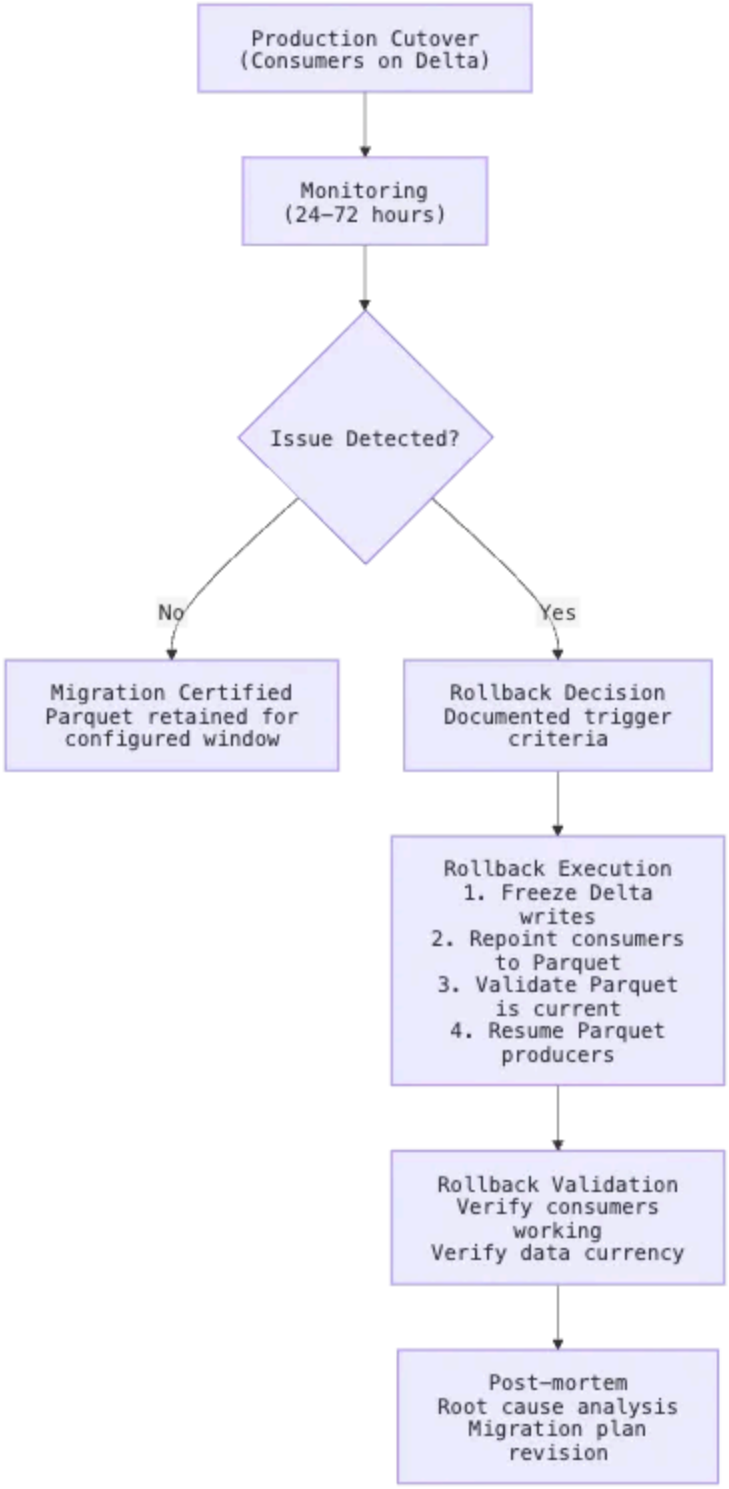
```
-- Rename the new Delta table to the production name
ALTER TABLE catalog.schema.customer_transactions_delta
RENAME TO catalog.schema.customer_transactions;

-- Option 2: Update table properties to track state
ALTER TABLE catalog.schema.customer_transactions
SET TBLPROPERTIES ('migration.status' = 'cutover_complete');
```

For consumers using hard-coded paths (which you will find in every mature environment), the cutover requires updating those references. This is not an automated operation — it requires finding every reference and updating it. This is one of the main reasons the dependency analysis in Section 6 is so important.

16. Rollback Strategy

Rollback is a first-class requirement, not an afterthought. Before you migrate a single table, you must have a documented and tested rollback plan for it.



What Constitutes a Rollback Trigger

Define this before cutover, not during an incident. Examples:

- Row count diverges by more than 0.1% within 24 hours of cutover
- A consumer job produces incorrect aggregations

- A business owner flags data quality issues in reporting
- Performance regression exceeds agreed threshold (e.g., 30% slower)
- A downstream system reports data errors

Rollback Mechanics

If using CTAS (Parquet unchanged): Rollback is straightforward. The original Parquet table was never touched. Repoint consumers. Stop the Delta producer. Resume the Parquet producer.

If using CONVERT TO DELTA: This is harder. The Parquet files were converted in-place. You can "roll back" by removing the `_delta_log` directory, but this is risky and may not be atomic. For CONVERT TO DELTA scenarios, maintain a metadata backup of the pre-conversion state and consider whether this conversion approach is appropriate for tables that require clean rollback.

Data divergence during rollback: If new data landed in Delta after cutover and before rollback, and you roll back to the Parquet path, that new data doesn't automatically appear in Parquet. You must handle this explicitly — either by extracting the new Delta data and writing it to the Parquet path, or by accepting a brief data gap with a defined recovery process.

Retention Window

Retain the Parquet data for a minimum of:

- Low-risk tables: 7 days post-certification
- Medium-risk tables: 30 days post-certification
- High-risk tables: 60–90 days post-certification

- Critical tables: 90+ days, or until the next major business cycle completes

Storage cost is real. Balance retention against risk. For low-risk tables, 7 days is reasonable. For a critical financial table, 90 days may not be enough.

17. The Migration Failed — What Actually Happened?

Let me describe the failure scenarios that are realistic enough to plan for. These are not hypothetical.

Scenario 1: A Downstream Job Reads the Parquet Path Directly

After cutover, a Databricks job that runs nightly reads from `abfss://container@storage.dfs.core.windows.net/raw/orders/` rather than `catalog.raw.orders`. When the table was converted, the catalog was updated, but the job wasn't. The job continued reading old Parquet data, produced stale results, and nobody noticed for three days because the data didn't look wrong — it was just slightly out of date.

Lesson: Dependency discovery must include storage path scanning, not just catalog table reference scanning.

Scenario 2: A Producer Job Truncates the Delta Table on First Run

A pipeline using partition overwrite mode was migrated to Delta without updating the write logic. The first production run after cutover used `mode("overwrite")` without `replaceWhere`, which in Delta means "overwrite the entire table." Thirty days of production data were overwritten in a single job run.

Time travel rescued the data — `RESTORE TABLE` recovered the previous version within minutes. But the root cause was a write pattern review failure.

Lesson: Every producer job must be reviewed for write pattern compatibility. Do not assume that swapping “parquet” for “delta” in a writer is sufficient.

Scenario 3: A Schema Change Breaks a Consumer

During CTAS migration, an engineer noticed an opportunity to clean up a column name (renamed `cust_id` to `customer_id`). The schema change was documented internally but not communicated to a downstream team's notebook, which used `cust_id` explicitly. The notebook failed at runtime with a `ColumnNotFound` exception.

Lesson: During migration, do not opportunistically change schemas unless you have validated all consumers. Schema cleanup is a separate effort from format migration.

Scenario 4: Streaming Checkpoint Incompatibility

A Structured Streaming job was reading from a Parquet table using `readStream.format("parquet")`. After migration to Delta, the team updated the format to `delta` but kept the existing checkpoint. The streaming job failed to restart because the checkpoint contained offsets in a format incompatible with the Delta reader.

The checkpoint had to be deleted and the stream replayed from the beginning — a recovery that took four hours and required coordination with the downstream team.

Lesson: Streaming checkpoint migration is a separate concern from table migration. Plan it explicitly. Always have a replay strategy before touching streaming dependencies.

Scenario 5: Row Counts Match, Business Data Doesn’t

Validation showed row counts matching within 0.001%. The structural validation passed. The migration was certified.

Two weeks later, a finance analyst noticed that monthly revenue totals in a report were off by $47,000. Investigation revealed that the CTAS migration had silently applied a type cast from `DECIMAL(15,4)` to `DOUBLE` on a revenue column (because the Hive Metastore had registered the column as `DOUBLE` despite the actual Parquet files storing `DECIMAL`). Floating-point arithmetic on revenue figures introduced rounding errors that were invisible in row-count validation.

Lesson: Validate numerical columns with sum and aggregate comparisons, not just row counts. For financial or regulated data, add business-specific reconciliation checks.

Scenario 6: An External Application Reads Parquet Files Directly

An internal microservice — outside the Databricks ecosystem — used the Azure Data Lake SDK to read Parquet files directly from the storage path. It had no dependency on the Hive Metastore. When the table was converted to Delta in-place, the `_delta_log` directory appeared and the application's Parquet file reader began failing with unexpected file listing behavior.

Lesson: External consumers outside the Databricks ecosystem are invisible to catalog-level dependency analysis. You must scan for external application dependencies through infrastructure review, not just Databricks job analysis.

Scenario 7: A Rollback Creates Data Divergence

After cutover, a critical table showed validation anomalies and the team initiated a rollback. The rollback succeeded — consumers were repointed to the Parquet path. But during the 18 hours the Delta table was live, five

records had been inserted via a MERGE operation. Those records now existed in Delta but not in Parquet.

The team had to manually extract those five records from the Delta table and insert them into Parquet via a correction job. It worked, but it required emergency engineering effort at 11 PM.

Lesson: The rollback plan must explicitly address data written to Delta after cutover. This is the most operationally complex rollback scenario. Design for it explicitly.

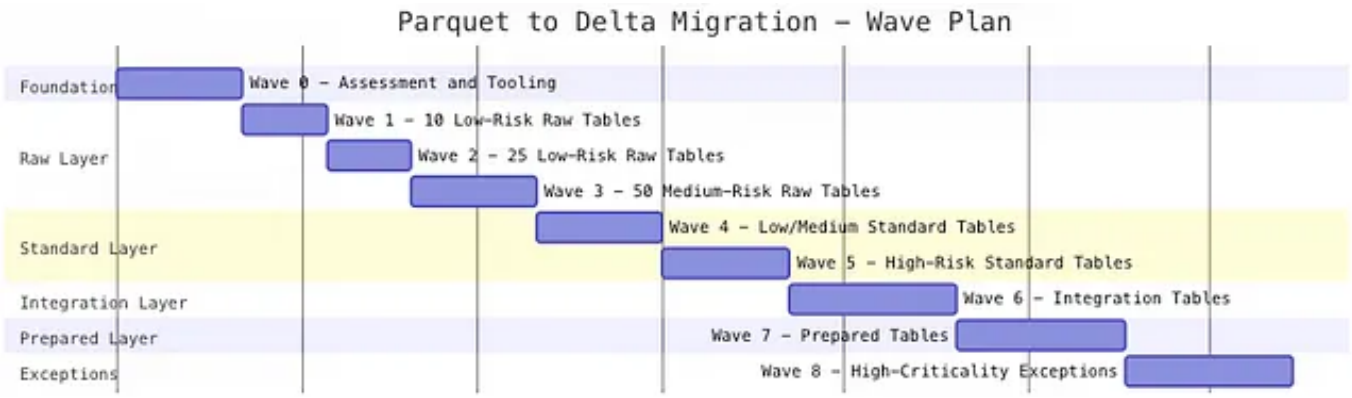
Scenario 8: A Supposedly Simple Table Had Hidden Dependencies

A small Raw table — 200 MB, single producer, no listed consumers, risk score 2 — was migrated in Wave 1 as a confidence-building exercise. Post-cutover monitoring revealed that an undocumented Azure Data Factory pipeline consumed this table and fed an external partner's SFTP. The partner's data feed went dark for 12 hours before the dependency was discovered.

Lesson: Dependency discovery is never complete. Low-risk tables still need discovery review before migration. Run a broader infrastructure scan, not just a Databricks job scan.

18. Migration Waves — A Realistic Rollout

The following wave structure is illustrative. Your actual wave design must be driven by the risk scores and dependency analysis from your specific environment.



Wave	Contents	Approx Tables	Goal
Wave 0	Assessment, tooling, framework validation	0	Build inventory, validate migration framework on sandbox
Wave 1	Low-risk Raw tables	10	First production migrations, framework validation
Wave 2	Low-risk Raw tables	25	Scale the process, identify tooling gaps
Wave 3	Medium-risk Raw tables	50	Handle partition complexity, larger tables
Wave 4	Low/medium Standard tables	50	Cross-layer dependencies, producer job reviews
Wave 5	High-risk Standard tables	75	Complex write patterns, CDC tables
Wave 6	Integration tables	150	Cross-layer coordination, MERGE patterns
Wave 7	Prepared tables	150	Consumer coordination, BI validation
Wave 8	High-criticality exceptions	30	Individual migration plans, extended validation

Total: ~540 tables in Waves 1–8 — the remaining ~300 are handled within the earlier waves.

Each wave should have:

1. A defined set of tables (from the inventory, by risk score and dependency order)
2. A Go / No-Go gate before cutover (validation results, stakeholder sign-off)
3. A rollback window (minimum 7 days post-cutover for Wave 1, scaling based on criticality)

4. A post-wave retrospective to identify issues before proceeding

Do not rush from Wave 1 to Wave 2 before Wave 1's rollback window expires. Patience in the early waves saves significant pain in the later ones.

19. After Migration — The Job Isn't Finished

Migration complete. Parquet tables decommissioned. This is where many teams declare victory and move on. It's also where the operational foundation of your Delta environment either gets established properly or becomes technical debt.

Unity Catalog Governance

If you haven't migrated to Unity Catalog already, a large-scale table migration is the right moment. Unity Catalog provides:

- **Column-level security:** Grant access at the column level, not just the table level
- **Row-level filters:** Dynamic data masking based on user identity
- **Automatic lineage:** Table-level and column-level lineage tracked automatically for all Delta tables
- **Centralized governance:** One policy model across all tables, all workspaces, all regions
- **Audit logs:** Complete access audit via system tables

```
-- Register table in Unity Catalog
CREATE TABLE main.production.customer_transactions
USING DELTA
LOCATION 'abfss://container@storage.dfs.core.windows.net/production/customer_tra
```

```
COMMENT 'Customer transaction fact table — migrated from Parquet 2024-03-15';

-- Grant fine-grained permissions
GRANT SELECT ON TABLE main.production.customer_transactions TO `data-analysts-gr
GRANT MODIFY ON TABLE main.production.customer_transactions TO `pipeline-service
```

OPTIMIZE and VACUUM Strategy

Establish a scheduled maintenance strategy for every Delta table:

```
-- OPTIMIZE for each table based on its write frequency
-- Daily for frequently written tables
OPTIMIZE catalog.schema.customer_transactions
ZORDER BY (customer_id, transaction_date);

-- VACUUM to remove old Parquet files
-- After rollback window expires
VACUUM catalog.schema.customer_transactions RETAIN 168 HOURS; -- 7 days
```

Important: `VACUUM` With a retention period shorter than 7 days is supported only if you explicitly disable the safety check. Don't do this unless you know exactly why you need to.

Cost Management

Delta tables generate `_delta_log` files and checkpoint files. Monitor storage growth beyond just data growth. Establish VACUUM schedules. Monitor the number of small files in each table using:

```
DESCRIBE DETAIL catalog.schema.table;
-- Check 'numFiles' — if this is growing unboundedly, OPTIMIZE is needed
```

20. What I Would Do

I have covered the framework, the techniques, the failure scenarios, and the governance. Now let me be direct: if I owned this platform with 840 Parquet tables, here is exactly how I would approach this migration.

1. Stop all new Parquet table creation immediately. From this point forward, all new tables are Delta. This is not negotiable. Every new Parquet table created during a migration program is debt that makes the migration harder.

2. Build the metadata inventory in the first three weeks. Automate what you can (Hive Metastore queries, job definition scans, path references). Then spend a week doing manual discovery with pipeline owners. The conversations are uncomfortable. They are also the most valuable part of the entire program.

3. Risk-score every table and assign migration waves. Don't start migrating until the inventory is at least 80% complete and all high-risk tables are identified. Starting migration before the inventory is ready guarantees surprises.

4. Start Wave 1 with tables you don't care about. I would pick 10 Raw tables that are small, immutable, and have no downstream consumers. Use these to validate the migration framework, the validation tooling, the rollback process, and the operational runbook. The goal is not to migrate tables — the goal is to build a migration machine.

5. Enforce a three-person review gate for every high-risk table. The migration engineer, the pipeline owner, and a data consumer representative must all sign off before cutover. This is not bureaucracy — it catches the downstream dependencies that automated discovery misses.

6. Treat write pattern review as mandatory, not optional. No producer job migrates to Delta without an explicit code review of its write logic. The Parquet-to-Delta writer change is a one-line code change. The write pattern review is where the real risk is.

7. Build the rollback plan before the migration plan for each table. For every table in Waves 5 and above, document the rollback procedure before migration begins. Not after. If you can't document a clean rollback, you're not ready to migrate.

8. Run at least one full business reporting cycle before decommissioning Parquet. For Prepared tables that feed BI tools and applications, retain the Parquet data until at least one monthly business reporting cycle has completed on Delta. Business users are the last line of validation.

9. Don't skip the performance benchmarks. For the top 50 most-queried tables, capture query performance before and after migration. Investigate regressions before they become production complaints.

10. Establish Delta operational hygiene on Day 1 of each wave. OPTIMIZE schedule, VACUUM schedule, data quality monitoring, table properties — these should be configured at the time of migration, not retrofitted later.

11. Dedicate one engineer to tracking migration state full-time. The migration inventory is a living database. It needs an owner. At 840 tables across multiple waves and months, migration state management is a part-time job for the team and a full-time tracking responsibility for one person.

12. Declare success only after the rollback window expires. A migration is not complete at cutover. It is complete when the rollback window closes, the

Parquet data is decommissioned, and the documentation is updated. Until then, it's in a certified-but-reversible state.

21. Final Takeaways

We set out with 840 Parquet tables and one question: *"Why don't we just convert them all to Delta?"*

By now, I hope the answer is clear. The question itself wasn't wrong. The assumption buried inside it — that this was a simple file-format conversion — was.

Here's what I want you to take from this:

The format conversion is the easy part. `CONVERT TO DELTA` takes thirty seconds. What takes weeks is understanding what every table does, who uses it, how it's written, and what breaks when it changes.

Dependencies are the real migration target. You're not migrating tables. You're migrating an ecosystem of producers, consumers, write patterns, and implicit assumptions.

Metadata is your foundation. You cannot manage a migration at this scale from memory or spreadsheets. Build the inventory. Maintain it. Drive every decision from it.

Risk-based sequencing is not optional. Migrating high-risk tables early, before your migration framework is validated, is how migrations fail visibly and expensively.

Delta is genuinely better for a production Lakehouse. Not because it's newer, but because ACID transactions, MERGE, schema enforcement, time travel, and Change Data Feed solve real production problems that Parquet cannot. The investment in migration is justified by the operational reliability you gain.

The job is not done at cutover. Delta requires operational governance. OPTIMIZE schedules, VACUUM policies, Unity Catalog permissions, data quality monitoring — these are ongoing responsibilities, not one-time setup.

And finally:

> *"Migrating Parquet to Delta is one of the most high-leverage investments a Databricks team can make in their platform. Just don't underestimate what you're actually migrating."*

The engineering is the easy part. The discipline, the process, the communication, and the patience — that's where migrations succeed or fail.

Databricks + Big Data + Data Engineering + Data + Apache Spark +



Written by Anurag Sharma

132 followers · 6 following

Edit profile

Data Engineering Specialist with 11+ Exp. Passionate about optimizing pipelines, data lineage, and Spark performance and sharing insights to empower data pros!